المدرسة العليا لأساتذة التعليم
التقني المحمدية
جامعة الحسن الثاني بالدار البيضاء

Master SDIA
ENSET Mohammedia

Rapport de projet

Gestion des incidents de sécurité

Spring Security, JWT et React

Module : **Sécurité des systèmes distribués**

Réalisé par :
Mahboub Marouane

Encadré par :
M. Abdelmajid BOUSSELHAM

5 novembre 2026

Table des matières

1 Introduction **3**
1.1 Objectifs . . . 3

2 Architecture et modèle de données **4**
2.1 Architecture générale . . . 4
2.2 Entités et relations . . . 4

3 Authentification et autorisation **6**
3.1 Authentification . . . 6
3.2 Rôles et protection . . . 6
3.3 Profil et expiration . . . 6

4 Fonctionnalités et interfaces **7**
4.1 Suivi des incidents . . . 7
4.2 Tableaux de bord . . . 7

5 Vérifications et résultats **10**

6 Mise en place et utilisation **12**
6.1 Environnement technique . . . 12
6.2 Démarrage . . . 12
6.3 Comptes locaux . . . 12
6.4 API REST . . . 12
6.5 Validation et erreurs . . . 13
6.6 Démonstration . . . 13

7 Annexe : captures détaillées des essais **14**

8 Conclusion **28**

Table des figures

2.1 Architecture générale de l'application 4
2.2 Relations principales entre les entités 5

3.1 Profil utilisateur et changement du mot de passe 6

4.1 Historique consulté par le créateur de l'incident 7
4.2 Tableau de bord de l'employé 8
4.3 Tableau de bord du technicien 8
4.4 Tableau de bord de l'administrateur 9

5.1 Refus d'accès à l'incident d'un autre employé 10
5.2 Refus de modification par un technicien non affecté 11
5.3 Validation des données lors de la création d'un incident 11

7.1 Capture Postman de l'inscription réussie 14
7.2 Capture Postman de la connexion réussie 15
7.3 Capture Postman de la création d'un incident 15
7.4 Capture Postman de la liste des incidents de l'utilisateur 16
7.5 Capture Postman du détail d'un incident 16
7.6 Capture Postman de la connexion du second employé 17
7.7 Création de l'incident 1 par employee2@example.com 17
7.8 Création de l'incident 2 par employee1@example.com 18
7.9 employee2@example.com consulte uniquement l'incident 1 18
7.10 employee1@example.com consulte uniquement l'incident 2 19
7.11 Capture Postman des statistiques administrateur 19
7.12 L'administrateur consulte les deux incidents 20
7.13 L'administrateur consulte les rôles et identifiants des utilisateurs 20
7.14 L'administrateur affecte l'incident 1 au technicien 4 21
7.15 Le technicien consulte l'incident 1 qui lui est affecté 21
7.16 Le technicien passe l'incident 1 à IN_PROGRESS avec une note 22
7.17 Le technicien passe l'incident 1 à RESOLVED avec une note de traitement 22
7.18 Capture Postman de la résolution de l'incident par le technicien 23
7.19 Capture Postman de l'incident résolu visible par son créateur 23
7.20 Tableau de bord de l'employé avec le résumé de ses incidents 24
7.21 Accès à une API protégée sans JWT 24
7.22 Accès refusé avec un JWT invalide 25
7.23 Accès employé refusé à une route administrateur 25
7.24 Désactivation du compte par l'administrateur 26
7.25 Connexion refusée pour le compte désactivé 26
7.26 Requête refusée avec un JWT expiré 27
7.27 Inscription refusée pour un email existant 27

1. Introduction

Ce projet réalise une application de déclaration et de suivi des incidents informatiques. Il met en œuvre une API Spring Boot protégée par Spring Security et JWT, ainsi qu'une interface React. Le contrôle d'accès aux données est effectué côté serveur selon le rôle et les liens entre les utilisateurs et les incidents.

1.1 Objectifs

— Authentifier les utilisateurs et protéger les accès avec un jeton JWT à durée limitée.
— Séparer les droits des employés, techniciens et administrateurs.
— Conserver les affectations, changements de statut et notes de traitement.
— Fournir des tableaux de bord et vérifier les règles de sécurité avec Postman.

2. Architecture et modèle de données

2.1 Architecture générale

L'interface React communique en HTTP et JSON avec l'API Spring Boot. Après authentification, Axios transmet le jeton dans l'en-tête `Authorization: Bearer`. Le filtre JWT et Spring Security vérifient la requête avant son traitement. Les données sont stockées dans MySQL.

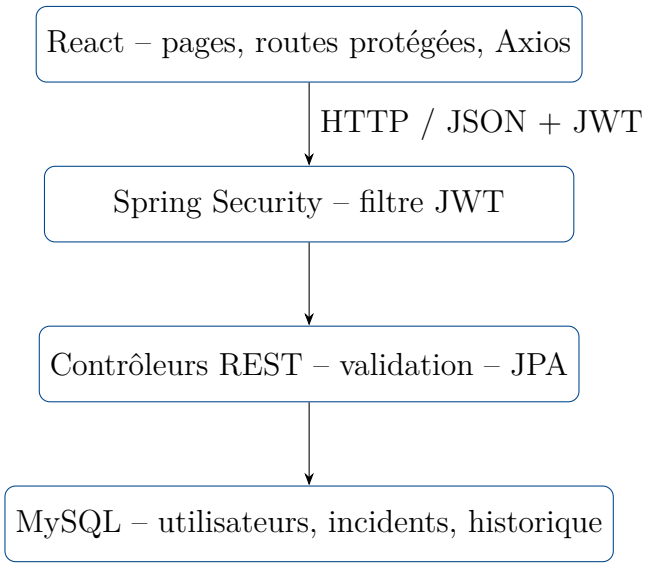


FIGURE 2.1 – Architecture générale de l'application

2.2 Entités et relations

L'entité `User` contient l'identité, l'email unique, le mot de passe haché, le rôle et l'état du compte. `Incident` contient le titre, la description, la catégorie, la priorité, le statut et les dates. Chaque incident possède un créateur et peut être affecté à un technicien. `IncidentHistory` conserve l'auteur, les états et une éventuelle note.

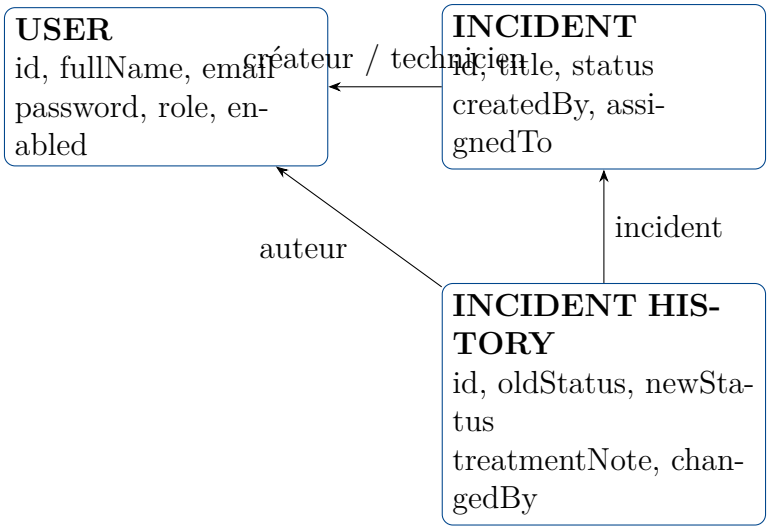


FIGURE 2.2 – Relations principales entre les entités

Les associations vers le créateur et le technicien sont des `@ManyToOne` chargées en mode `LAZY`. Les réponses DTO sélectionnent les données utiles et évitent la sérialisation récursive. Le diagramme détaillé est également disponible dans `docs/erd.md`.

3. Authentification et autorisation

3.1 Authentification

L'inscription publique crée un compte `EMPLOYEE` ; le rôle ne peut pas être choisi lors de cette opération. Le mot de passe est haché avec BCrypt. La connexion renvoie un JWT signé contenant l'email, le rôle, la date d'émission et la date d'expiration. L'API utilise une politique de session `STATELESS` et autorise le frontend local par CORS.

3.2 Rôles et protection

L'employé crée et consulte ses incidents. Le technicien consulte les incidents qui lui sont affectés et peut en modifier le statut avec une note. L'administrateur gère les comptes, consulte l'ensemble des incidents et les affecte. La propriété et l'affectation sont contrôlées par le backend. Les incidents fermés ne peuvent pas être modifiés par la mise à jour ordinaire.

3.3 Profil et expiration

Le profil permet de modifier le nom et l'email. Le changement de mot de passe exige l'ancien mot de passe et la confirmation du nouveau ; le serveur vérifie l'ancien avant l'enregistrement. Une réponse `401` sur une requête protégée déclenche la suppression de la session côté React et la redirection vers la connexion.

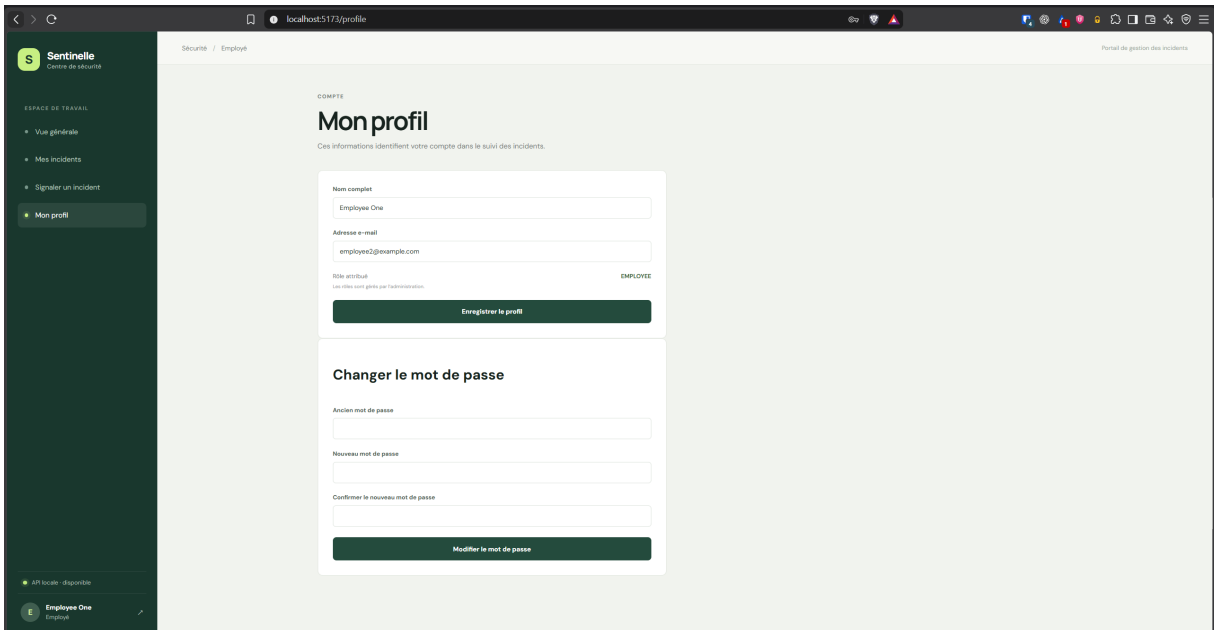
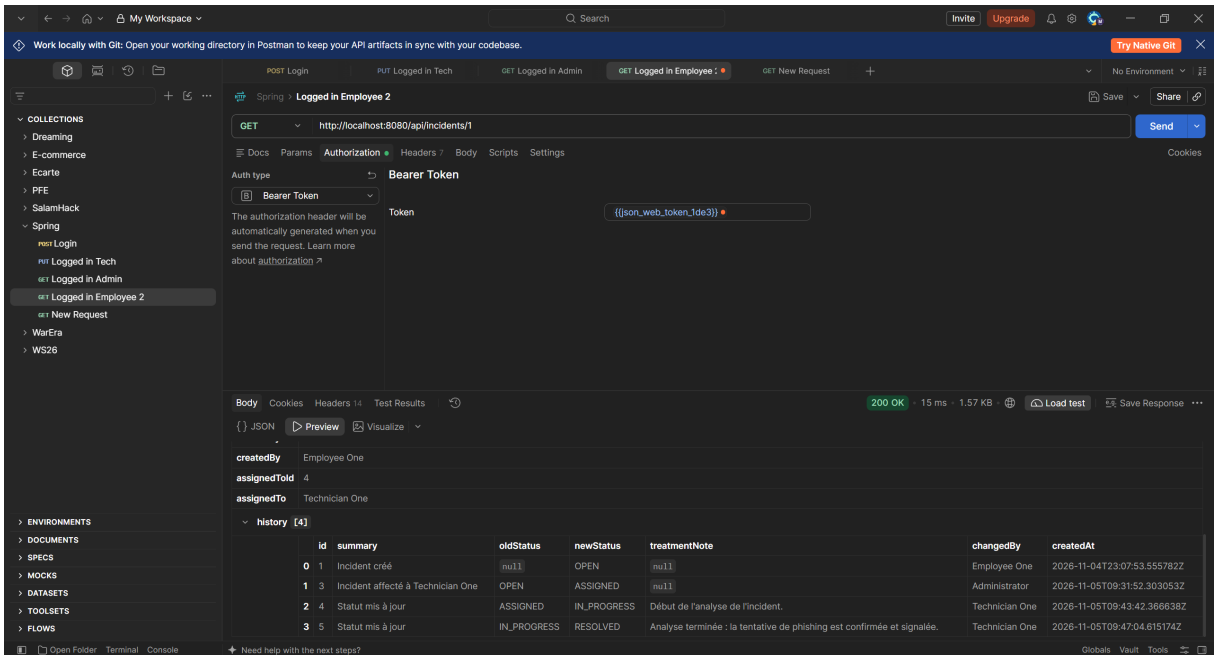


FIGURE 3.1 – Profil utilisateur et changement du mot de passe

4. Fonctionnalités et interfaces

4.1 Suivi des incidents

Un incident commence en état `OPEN`. L'affectation peut le faire passer à `ASSIGNED`, puis le traitement à `IN_PROGRESS` et `RESOLVED`. Un administrateur peut fermer un incident résolu. L'historique garde la trace des actions et des notes.



	id	summary	oldStatus	newStatus	treatmentNote	changedBy	createdAt
0	1	Incident créé	null	OPEN	null	Employee One	2026-11-04T23:07:53.555782Z
1	3	Incident affecté à Technician One	OPEN	ASSIGNED	null	Administrator	2026-11-05T09:31:52.303053Z
2	4	Statut mis à jour	ASSIGNED	IN_PROGRESS	Début de l'analyse de l'incident.	Technician One	2026-11-05T09:43:42.366638Z
3	5	Statut mis à jour	IN_PROGRESS	RESOLVED	Analyse terminée : la tentative de phishing est confirmée et signalée.	Technician One	2026-11-05T09:47:04.615174Z

FIGURE 4.1 – Historique consulté par le créateur de l'incident

4.2 Tableaux de bord

Les tableaux de bord présentent les indicateurs et incidents accessibles à chaque rôle : incidents personnels pour l'employé, incidents affectés et critiques pour le technicien, et statistiques globales pour l'administrateur.

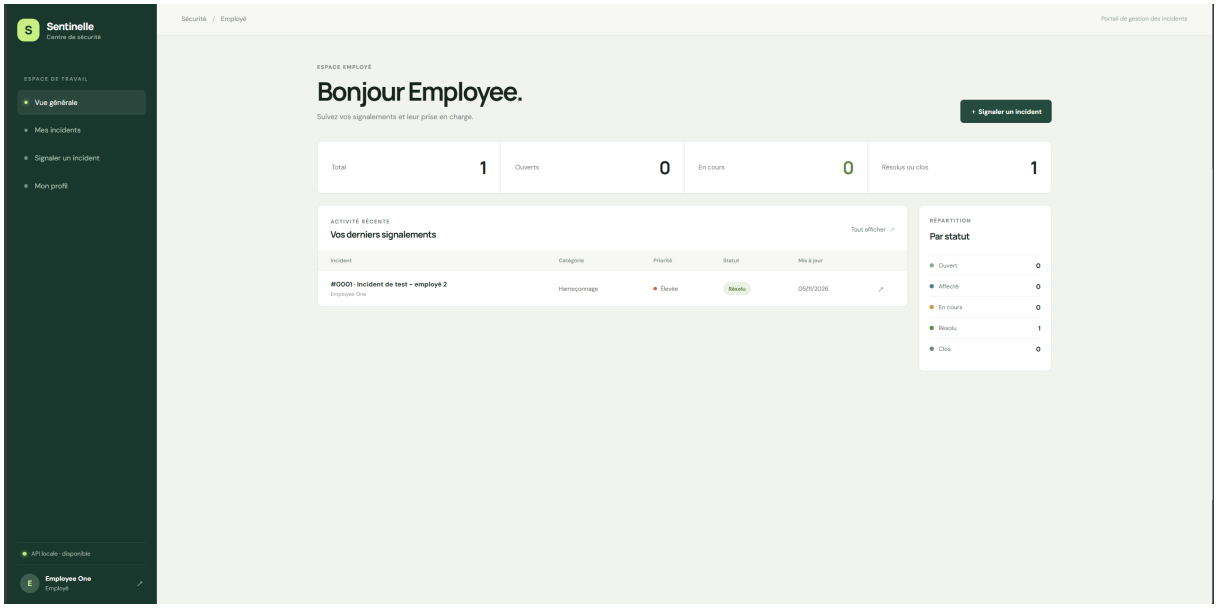


FIGURE 4.2 – Tableau de bord de l'employé

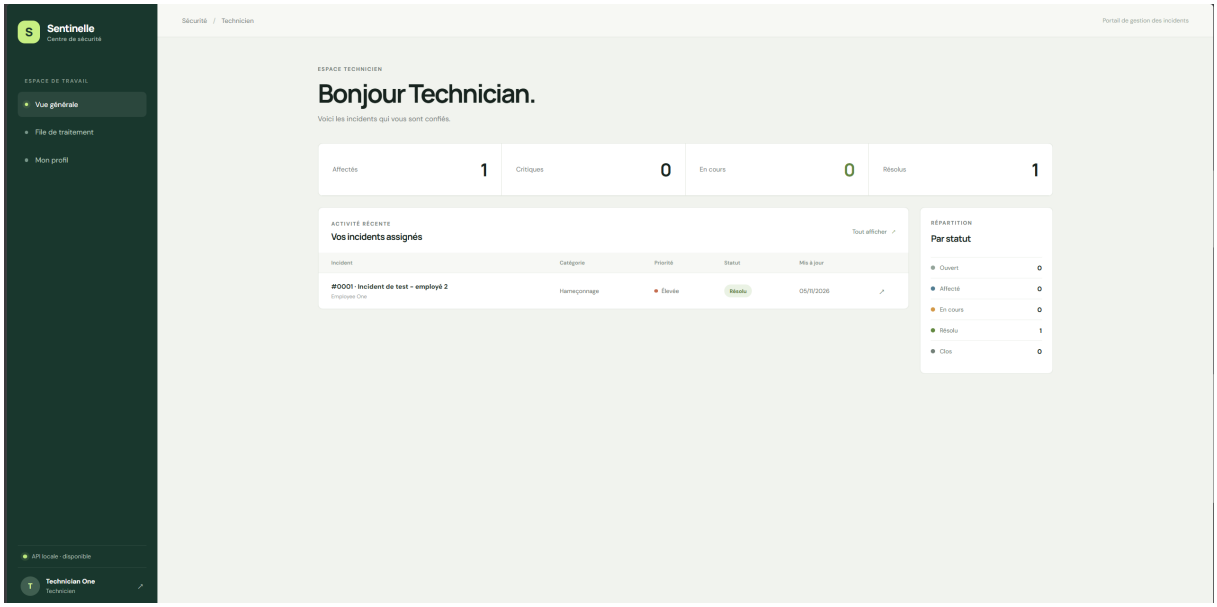


FIGURE 4.3 – Tableau de bord du technicien

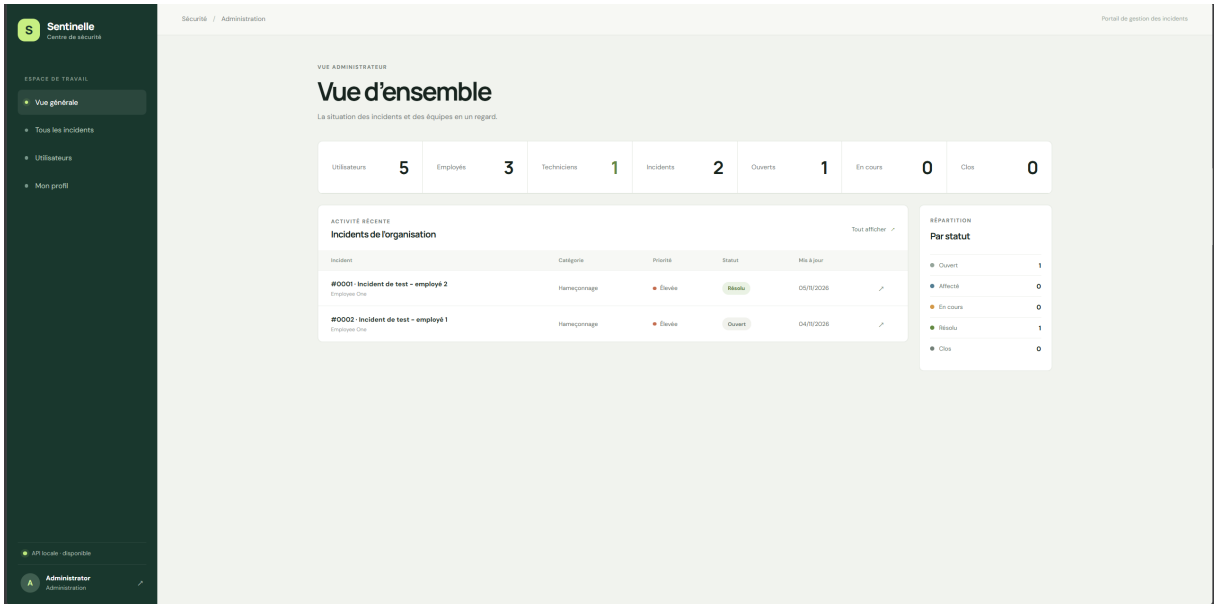


FIGURE 4.4 – Tableau de bord de l'administrateur

5. Vérifications et résultats

Les captures Postman du dépôt documentent le scénario de création, d'affectation, de traitement et de consultation de l'historique. Les neuf cas de sécurité demandés ont été exercés :

1. Requête sans JWT : `401`.
2. JWT invalide : `401`.
3. JWT expiré : `401` (jeton ancien de plus de trois heures, durée configurée d'une heure).
4. Employé sur une route administrateur : `403`.
5. Employé sur l'incident d'un autre : `403`.
6. Technicien sur un incident non affecté : `403`.
7. Connexion à un compte désactivé : `401`.
8. Inscription avec un email existant : `409`.
9. Incident avec données invalides : `400`.

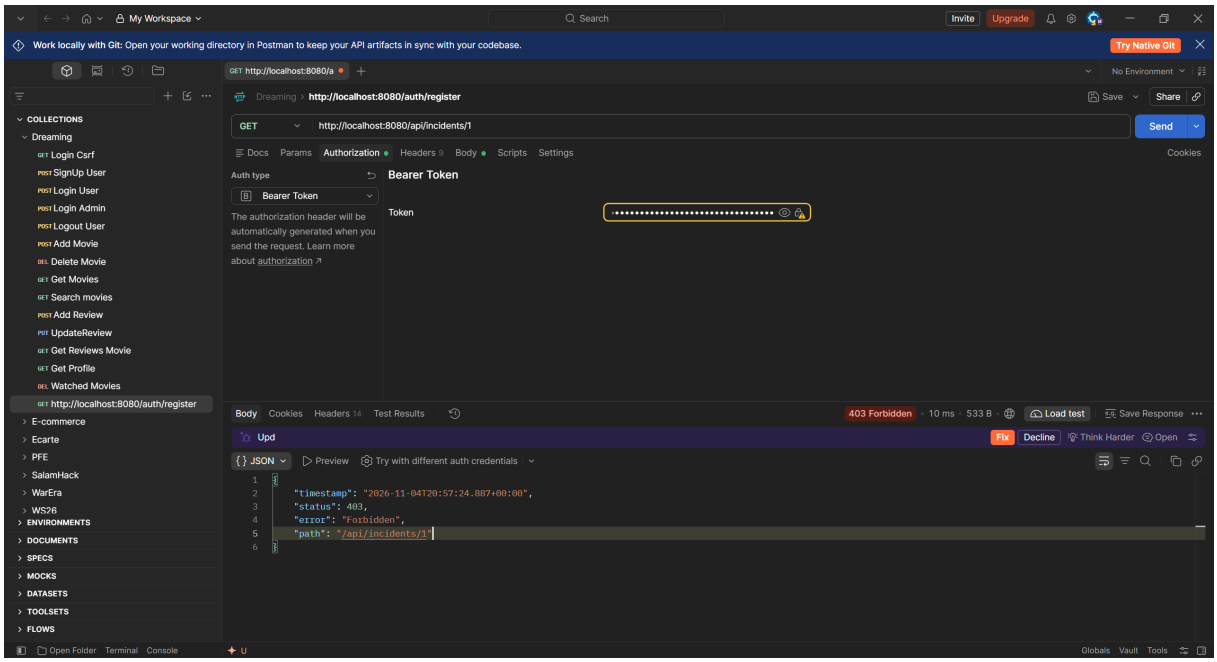


FIGURE 5.1 – Refus d'accès à l'incident d'un autre employé

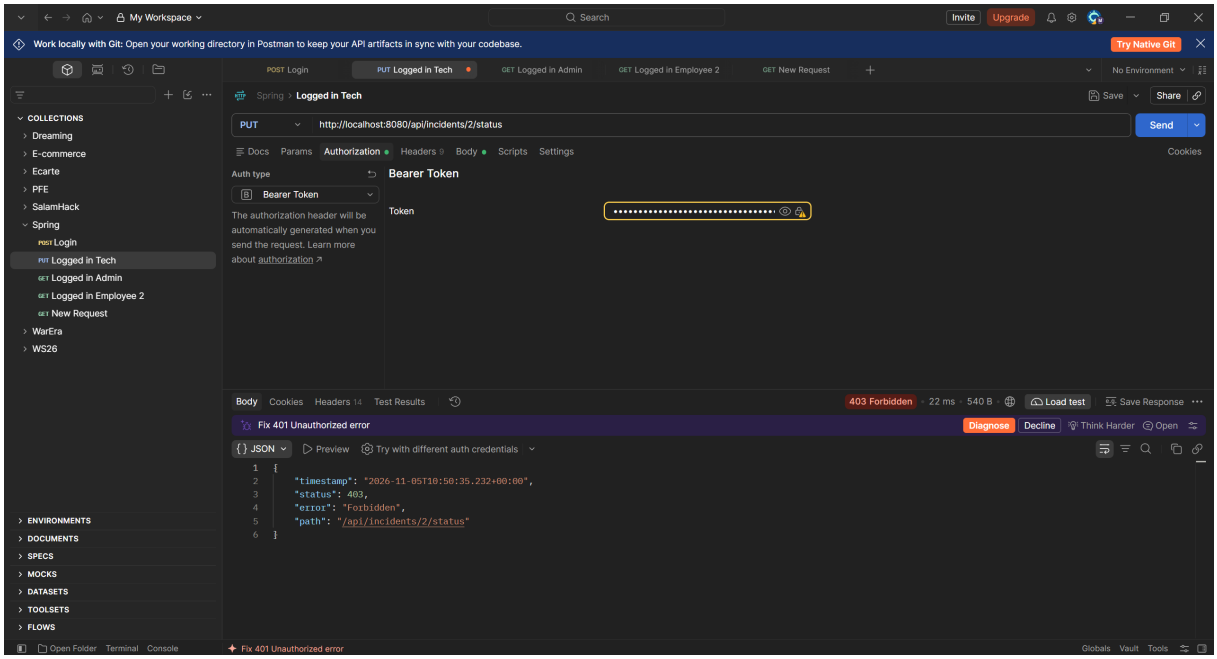


FIGURE 5.2 – Refus de modification par un technicien non affecté

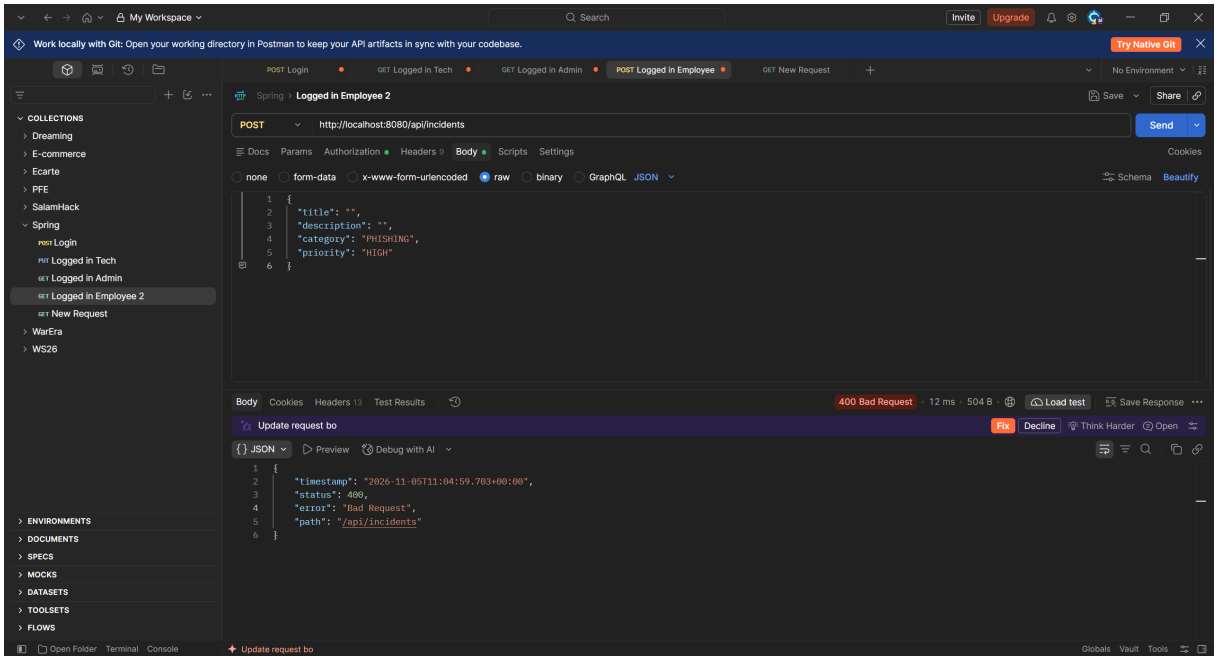


FIGURE 5.3 – Validation des données lors de la création d'un incident

Les captures détaillées figurent également en annexe et dans le README. Le scénario documenté utilise deux employés et un incident par employé ; pour la démonstration conforme à l'énoncé, le premier employé doit créer un second incident.

6. Mise en place et utilisation

6.1 Environnement technique

Le backend utilise Java 17 ou supérieur, Spring Boot, Spring Web, Spring Security, Spring Data JPA, Bean Validation et le pilote MySQL. Le frontend utilise React, Vite, React Router et Axios. MySQL 8.4 est démarré avec Laragon sur le port 3306. Le backend écoute sur le port 8080 et Vite sur le port 5173.

6.2 Démarrage

Créer la base `security_incidents`, démarrer MySQL, puis lancer `mvn spring-boot:run` dans `backend/`. Dans un second terminal, exécuter `npm install` puis `npm run dev` dans `frontend/`. Les variables `DB_URL`, `DB_USERNAME` et `DB_PASSWORD` permettent d'adapter la connexion. Le secret JWT peut être fourni par `JWT_SECRET`. Un premier administrateur peut être créé à l'aide des variables `BOOTSTRAP_ADMIN_EMAIL` et `BOOTSTRAP_ADMIN_PASSWORD`.

6.3 Comptes locaux

Les comptes ci-dessous sont ceux employés pour la démonstration locale ; ils ne sont pas créés automatiquement dans une nouvelle base. Les mots de passe correspondent aux données de démonstration utilisées lors des essais.

Rôle	Email	Mot de passe
Administrateur	admin@example.com	AdminDemo123 !
Employé 1	employee1@example.com	Employee123 !
Employé 2	employee2@example.com	Employee123 !
Technicien	tech1@example.com	Technician123 !

6.4 API REST

Les routes publiques sont `POST /auth/register` et `POST /auth/login`. Les routes suivantes exigent un JWT valide :

- `GET /api/profile`, `PUT /api/profile` et `PUT /api/profile/password` : consultation et mise à jour du profil et du mot de passe.
- `POST /api/incidents`, `GET /api/incidents/my` et `GET /api/incidents/{id}` : déclaration, liste personnelle et détail sous contrôle d'accès.
- `GET /api/incidents/assigned` et `PUT /api/incidents/{id}/status` : liste affectée et traitement par le technicien ; l'administrateur peut aussi changer le statut.
- `GET /api/incidents` et `PUT /api/incidents/{id}/assign/{technicianId}` : liste globale et affectation par l'administrateur.
- `GET /api/admin/users`, `PUT /api/admin/users/{id}/role`, `/enabled`, `/enable`, `/disable` et `GET /api/admin/statistics` : administration des comptes et statistiques.

6.5 Validation et erreurs

Les requêtes sont validées avec `@Valid`, `@NotBlank`, `@Email` et `@Size`. Une donnée invalide entraîne une réponse 400 ; une authentification absente ou invalide, 401 ; un accès interdit, 403 ; une ressource introuvable, 404 ; et un email déjà utilisé, 409. Le changement de mot de passe rejette l'ancien mot de passe incorrect et une confirmation différente.

6.6 Démonstration

Le scénario demandé commence avec deux employés et un technicien. Le premier employé déclare deux incidents et le second en déclare un. Il faut vérifier que le premier ne peut pas lire l'incident du second, puis utiliser le compte administrateur pour consulter et affecter les incidents. Le technicien consulte sa liste, passe un incident en cours puis résolu, et ajoute une note. Le créateur consulte enfin le statut et l'historique. Les captures existantes documentent un incident pour chaque employé : le deuxième incident du premier employé est à créer avant la démonstration complète.

7. Annexe : captures détaillées des essais

Les figures suivantes rassemblent les captures du README et complètent les explications des chapitres précédents. Les réponses et les dates indiquées correspondent aux essais effectués localement. Les jetons affichés dans certaines captures sont ceux des sessions de test.

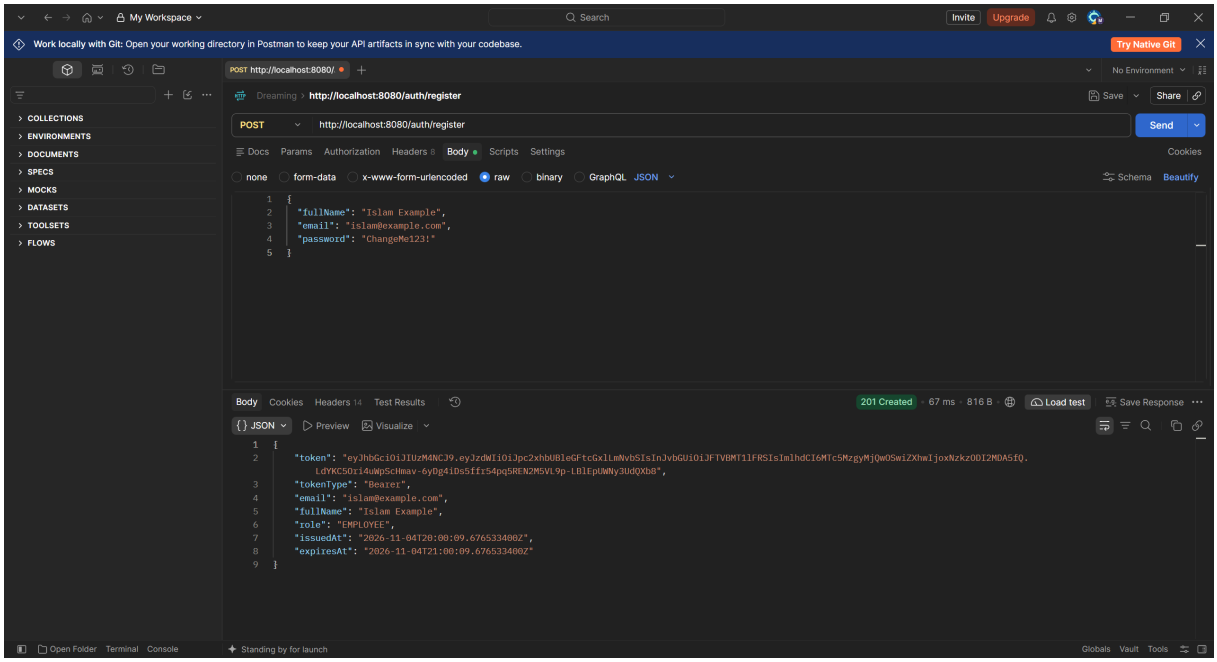


FIGURE 7.1 – Capture Postman de l'inscription réussie

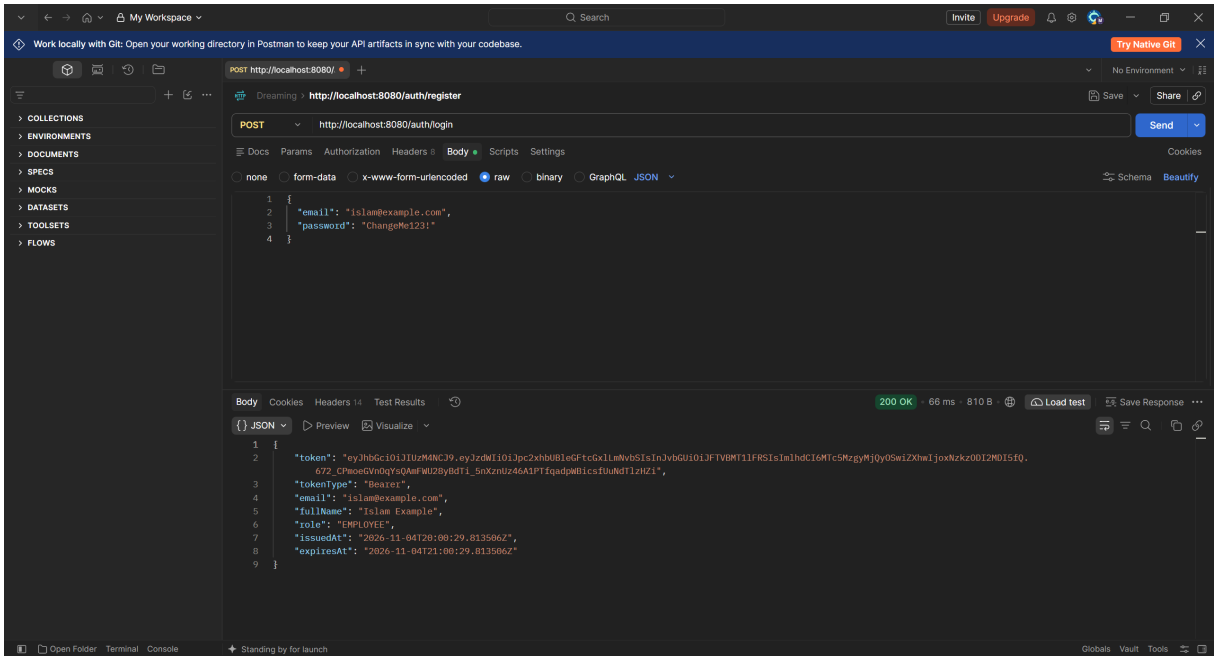


FIGURE 7.2 – Capture Postman de la connexion réussie

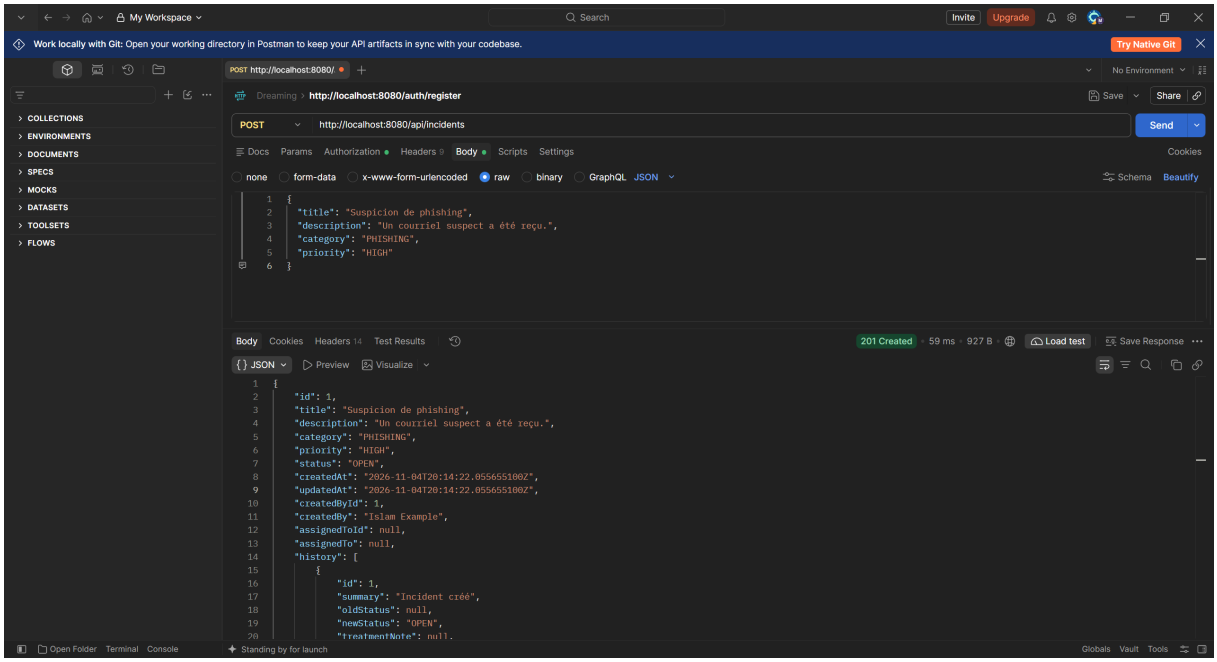


FIGURE 7.3 – Capture Postman de la création d'un incident

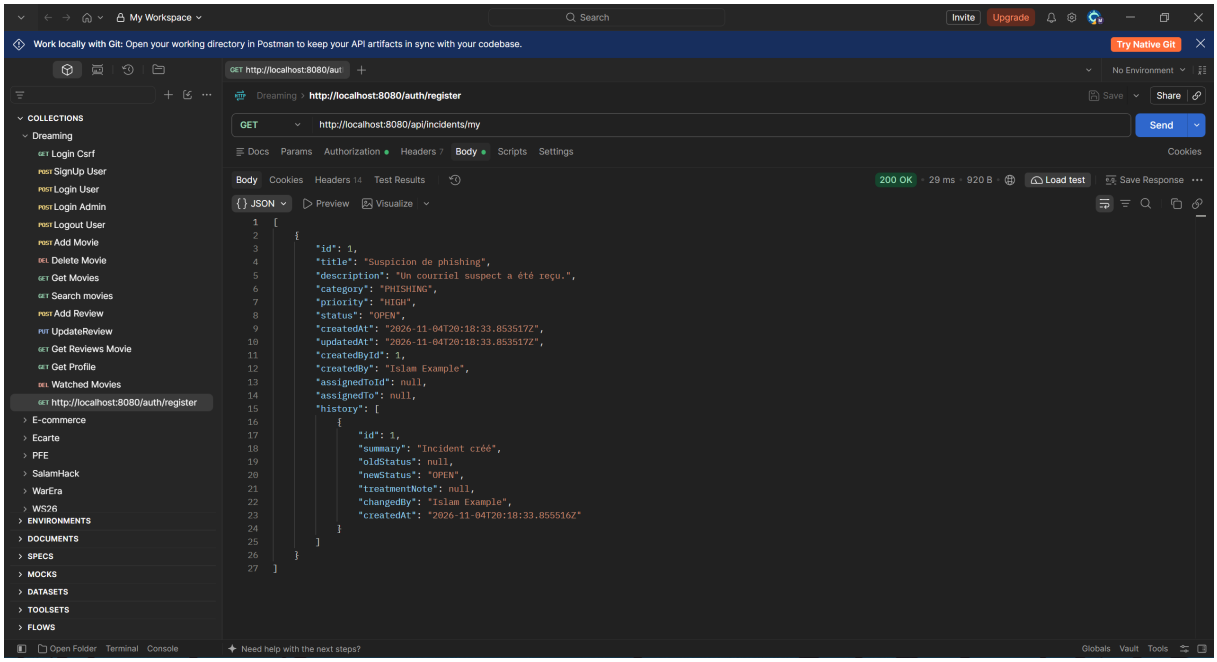


FIGURE 7.4 – Capture Postman de la liste des incidents de l'utilisateur

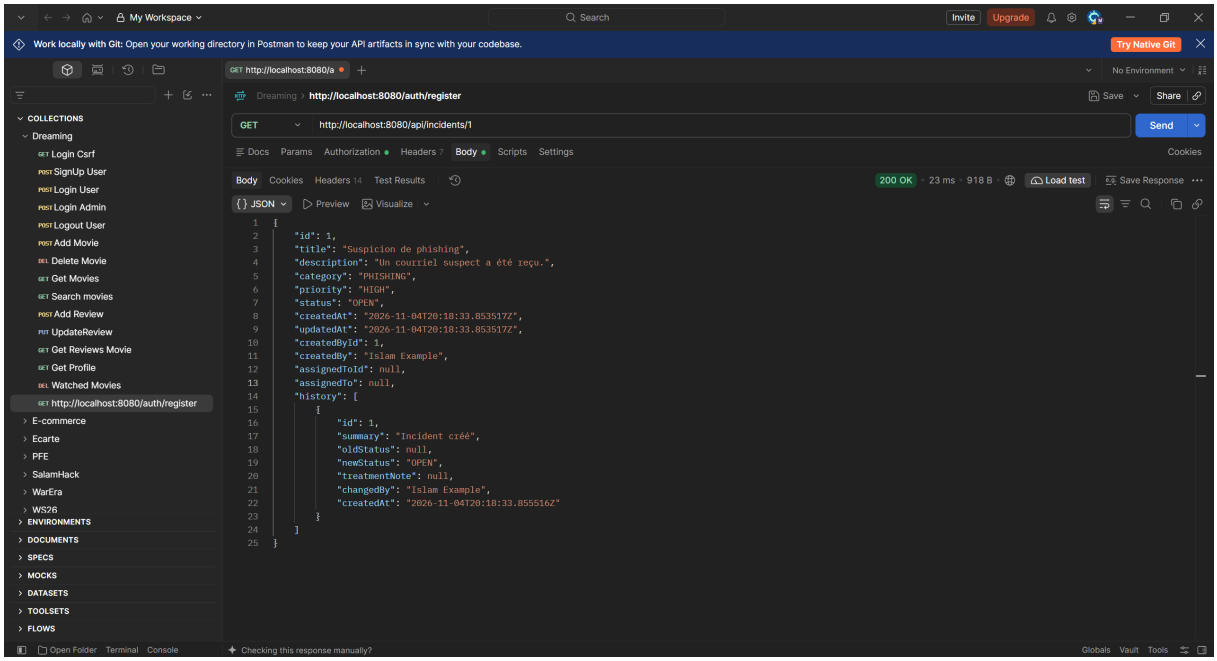


FIGURE 7.5 – Capture Postman du détail d'un incident

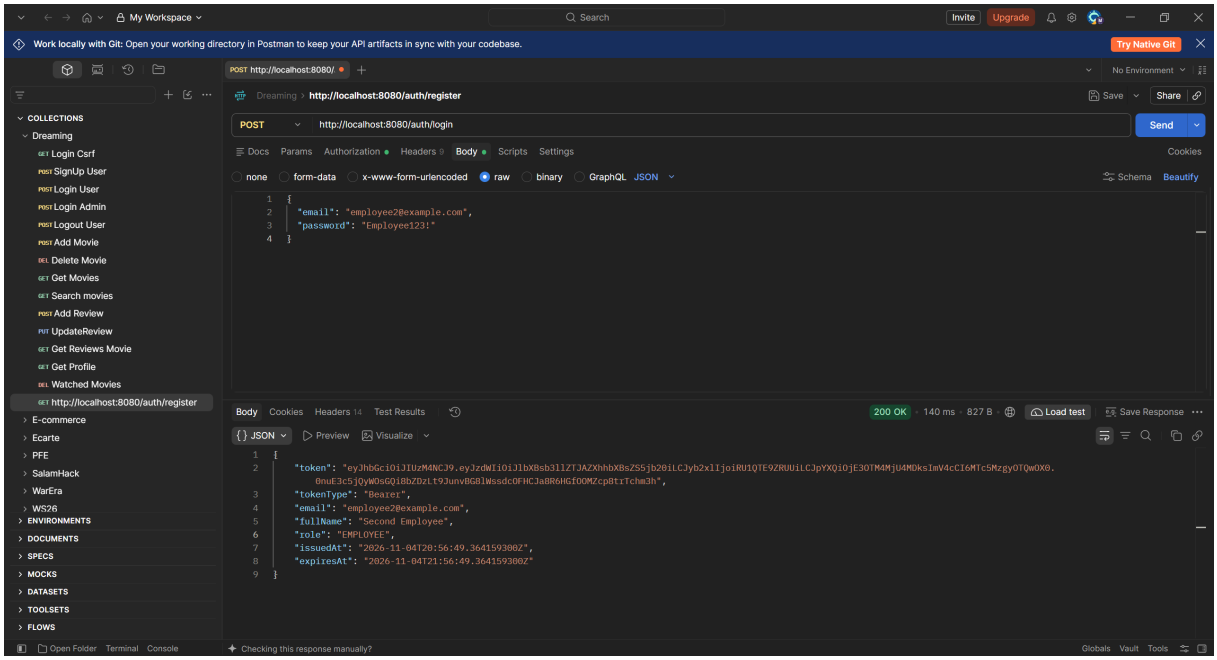


FIGURE 7.6 – Capture Postman de la connexion du second employé

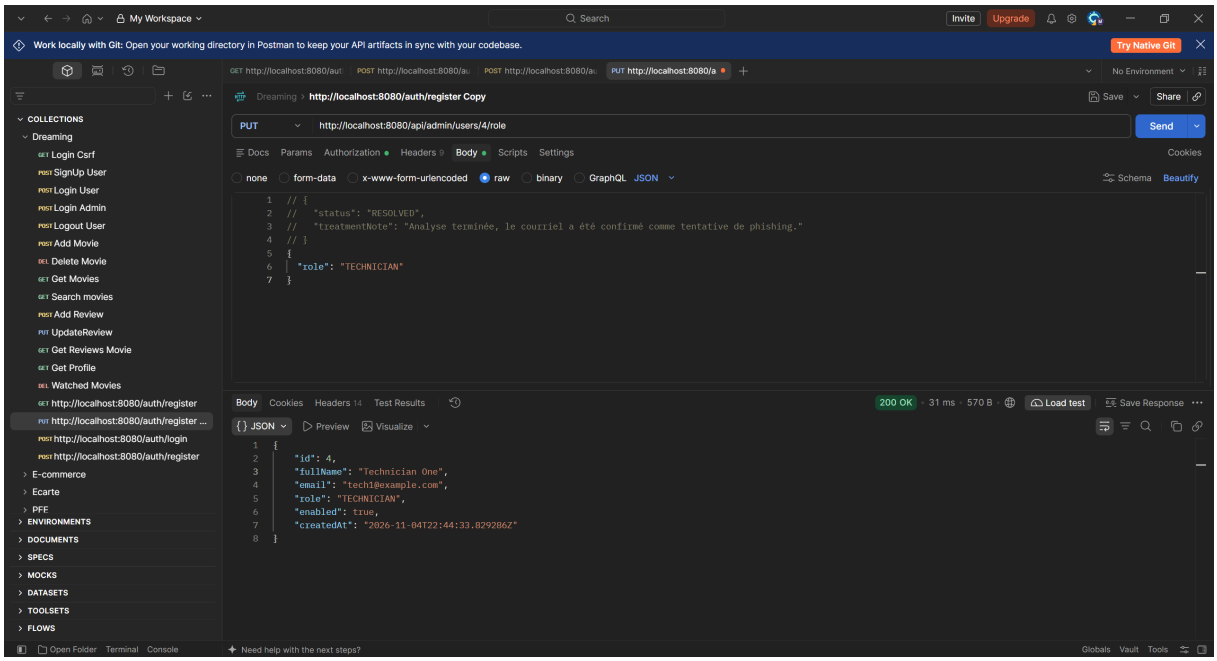


FIGURE 7.7 – Création de l'incident 1 par employee2@example.com

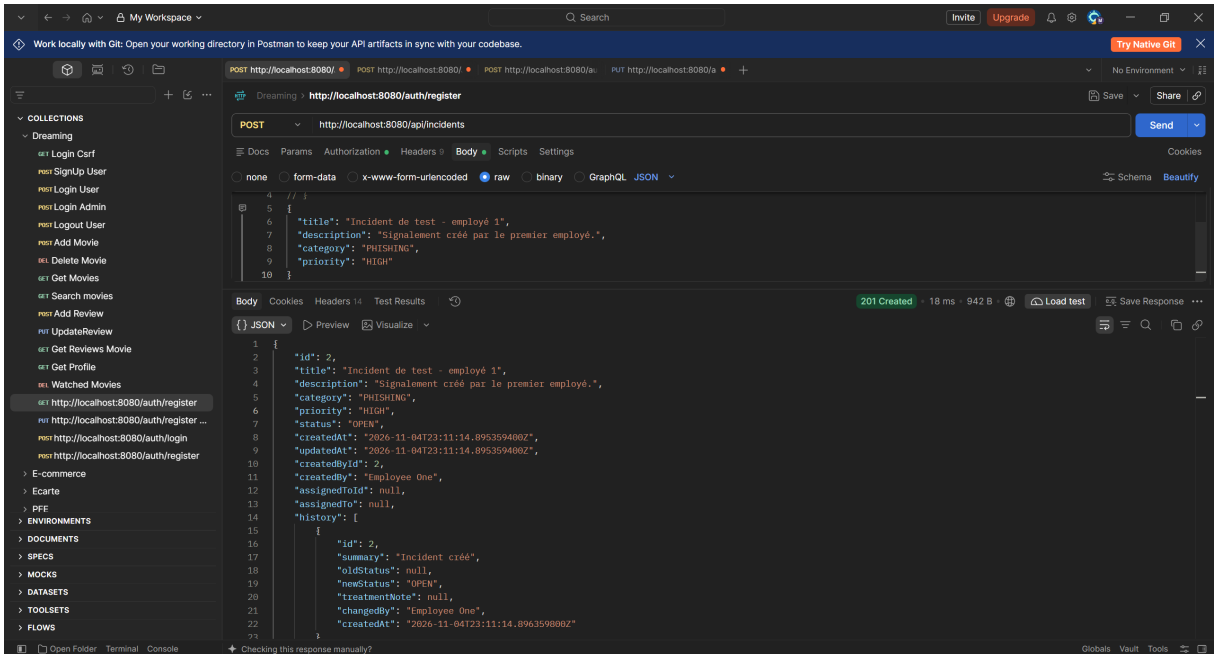


FIGURE 7.8 – Création de l'incident 2 par employee1@example.com

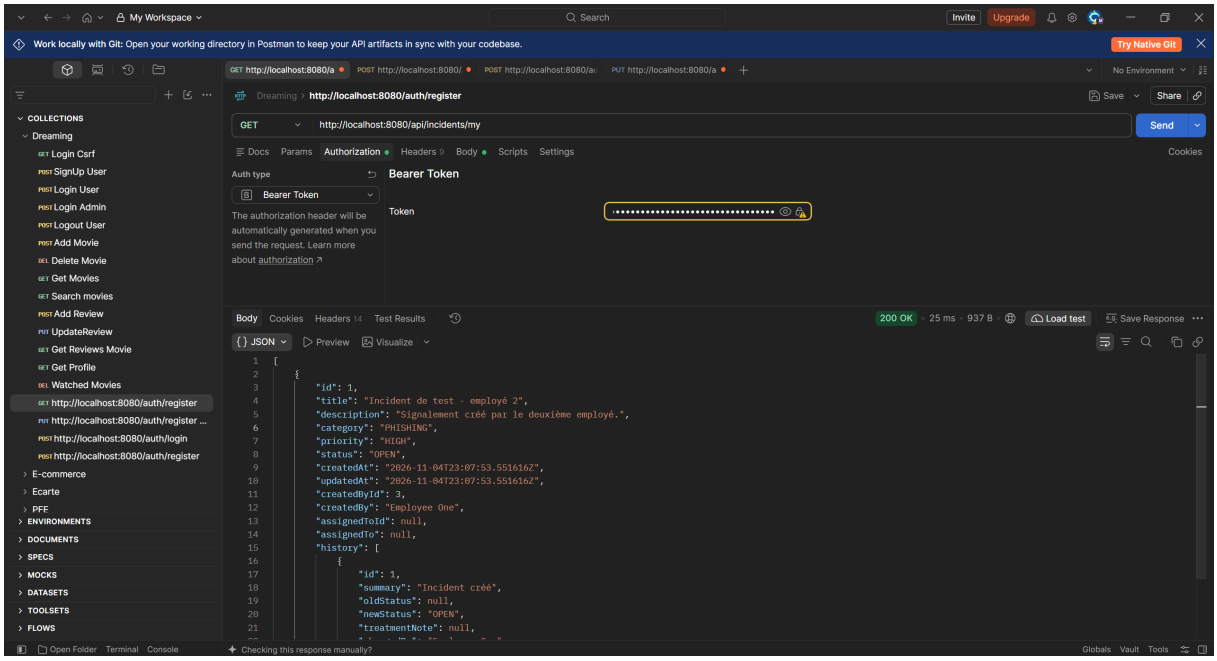


FIGURE 7.9 – employee2@example.com consulte uniquement l'incident 1

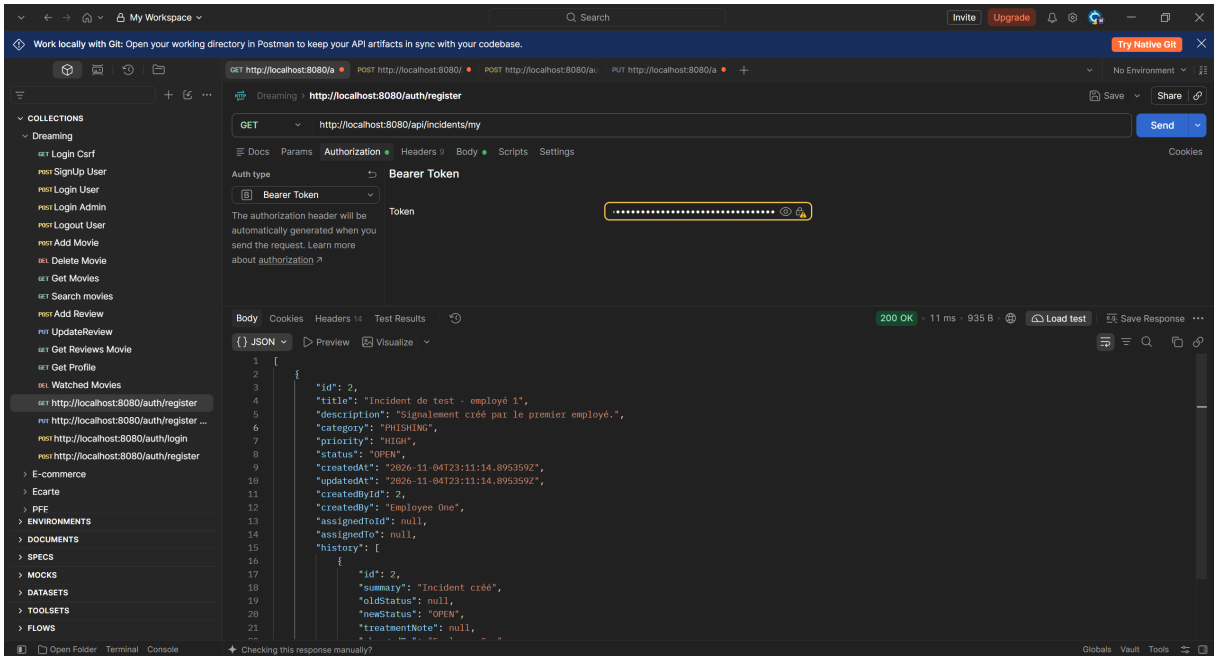


FIGURE 7.10 – employee1@example.com consulte uniquement l'incident 2

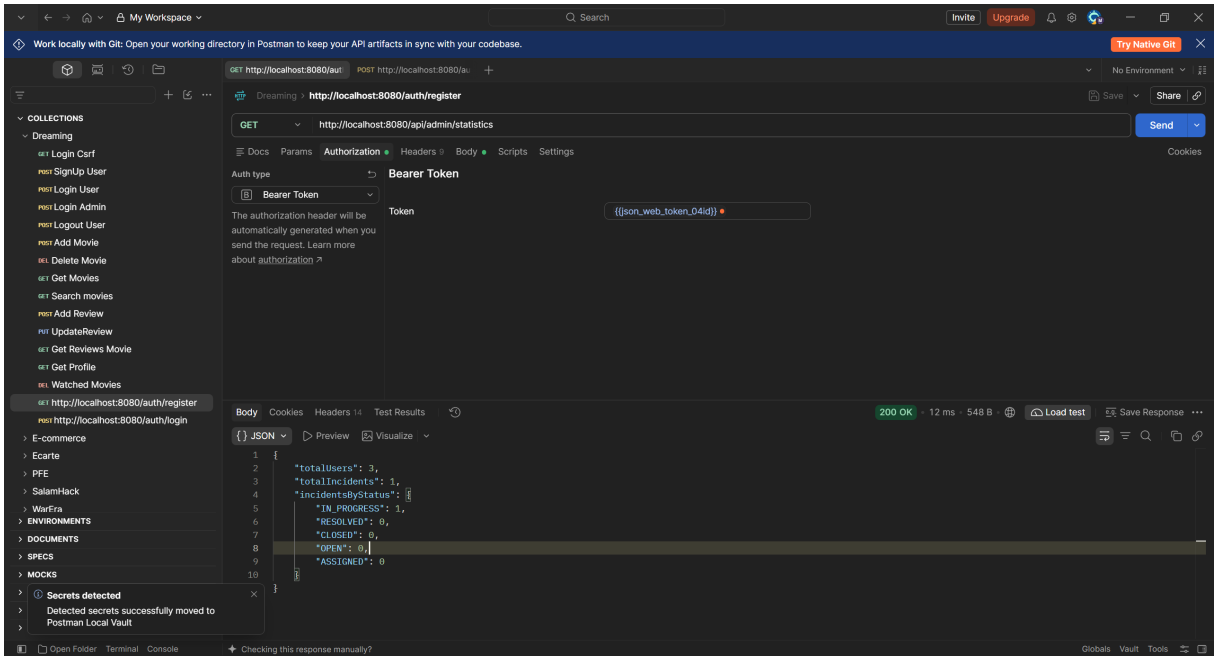


FIGURE 7.11 – Capture Postman des statistiques administrateur

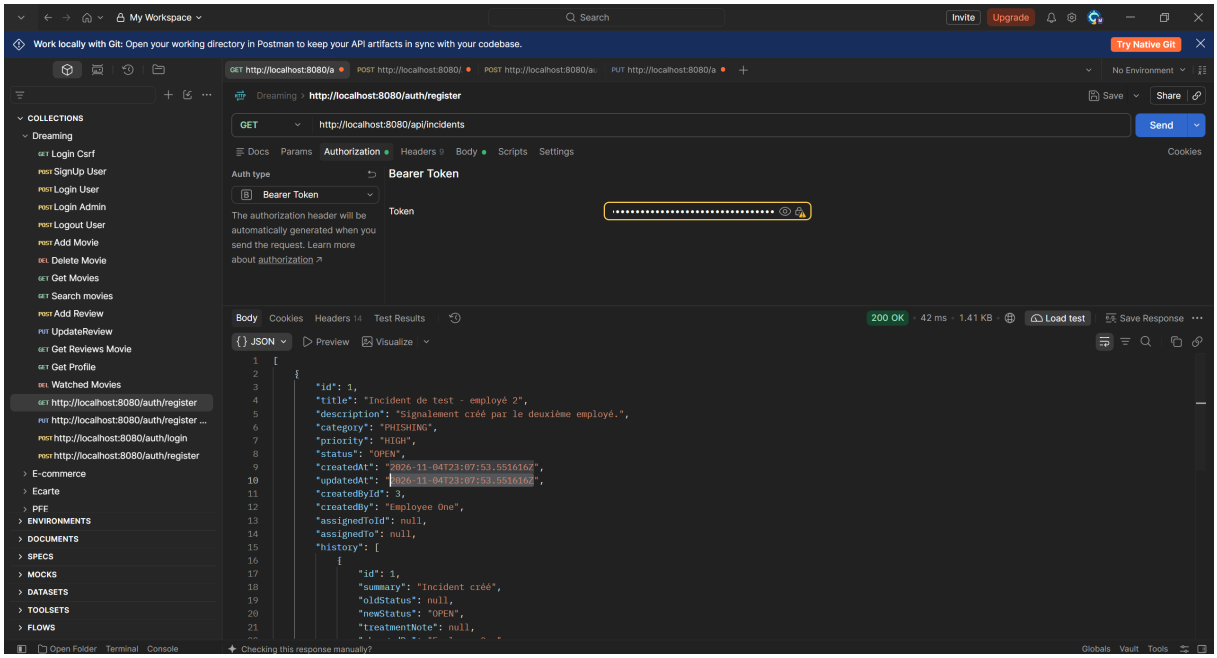


FIGURE 7.12 – L'administrateur consulte les deux incidents

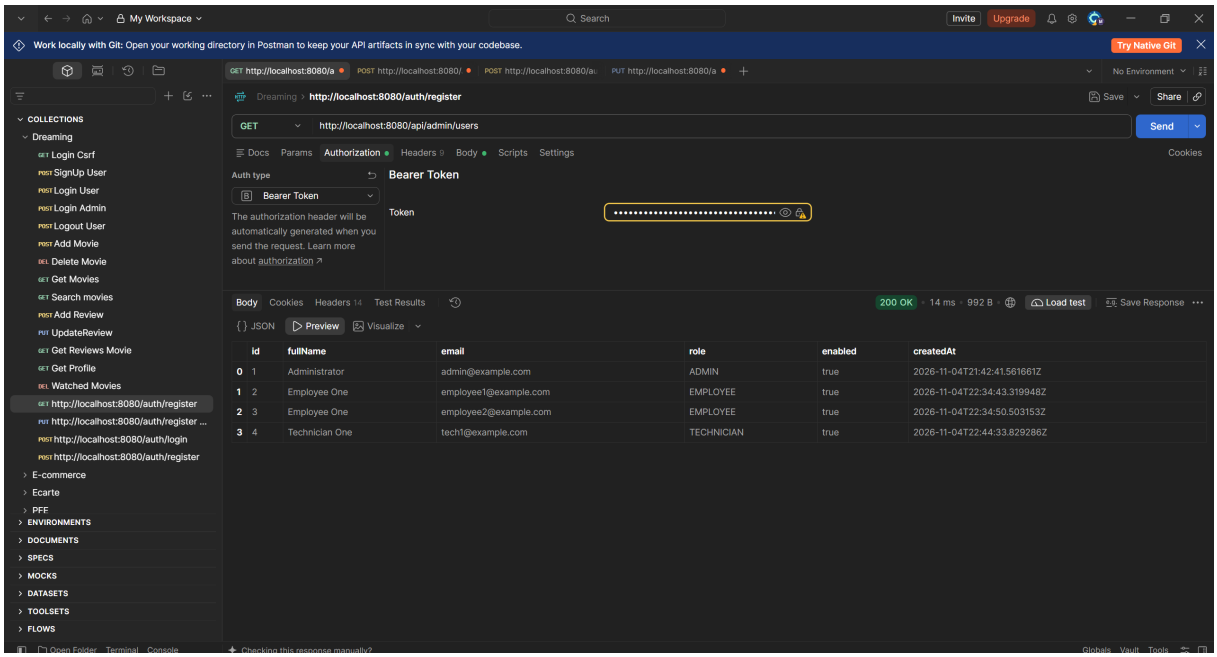


FIGURE 7.13 – L'administrateur consulte les rôles et identifiants des utilisateurs

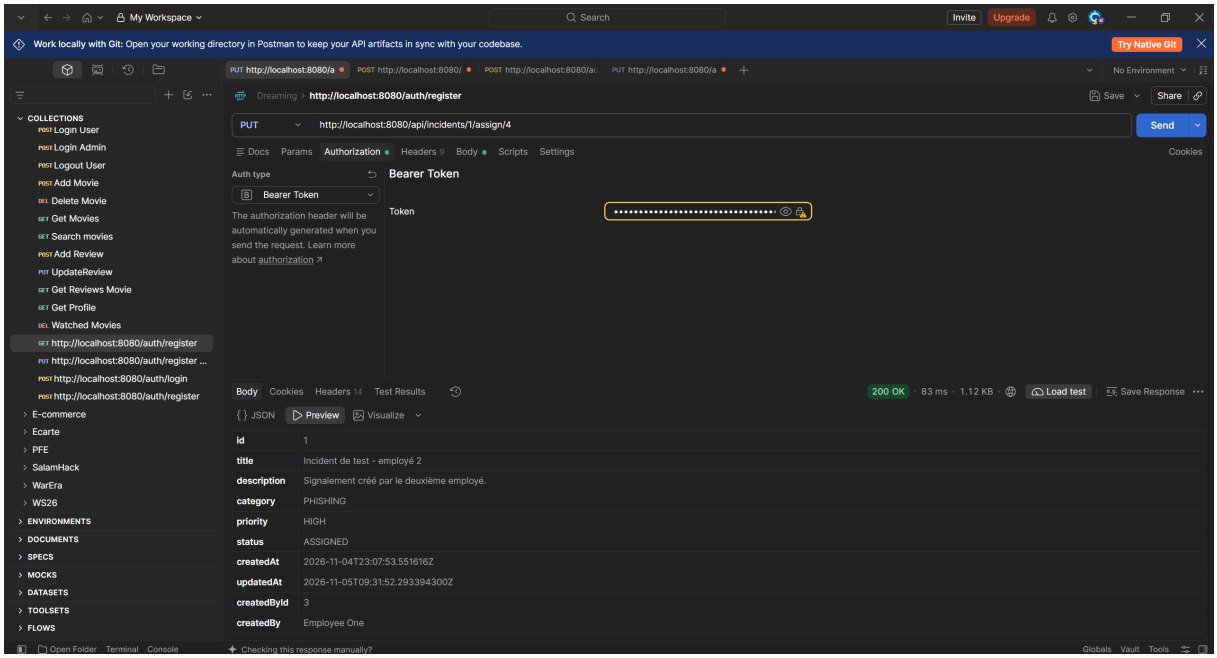


FIGURE 7.14 – L'administrateur affecte l'incident 1 au technicien 4

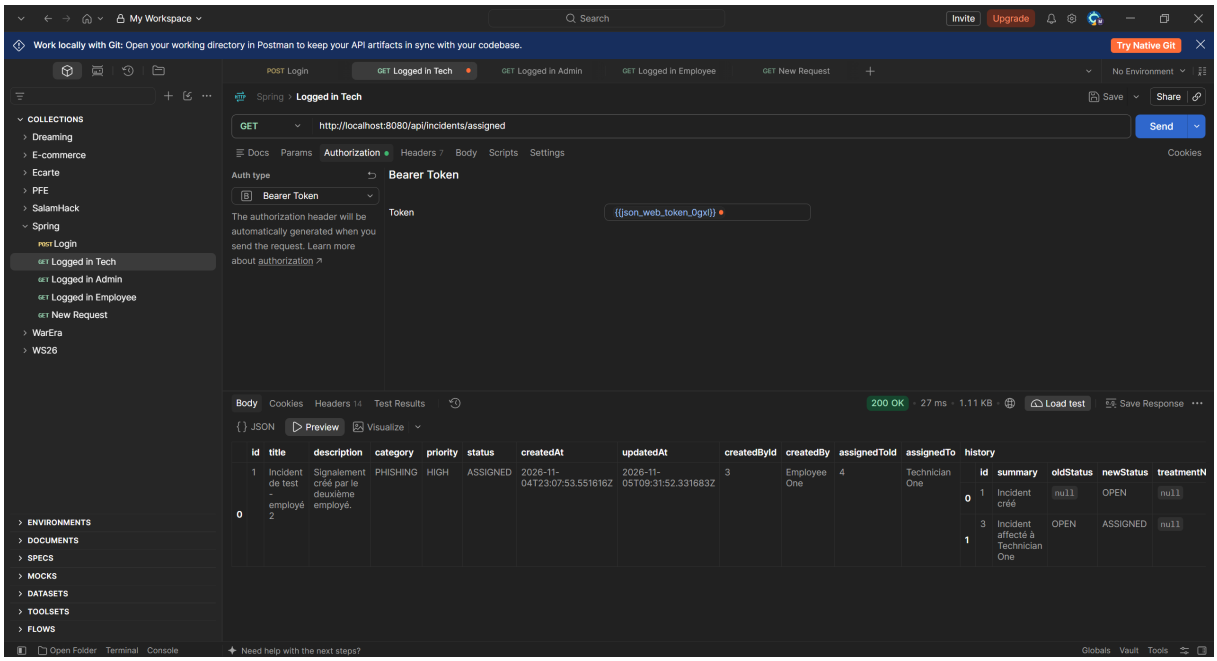


FIGURE 7.15 – Le technicien consulte l'incident 1 qui lui est affecté

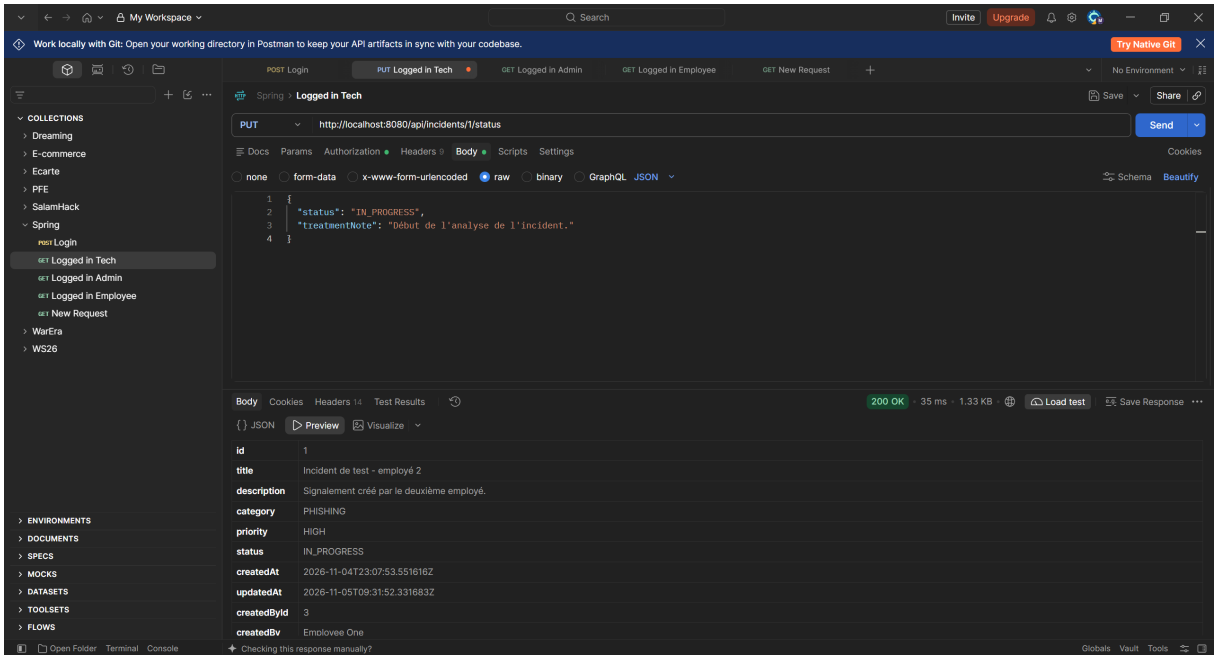


FIGURE 7.16 – Le technicien passe l'incident 1 à IN_PROGRESS avec une note

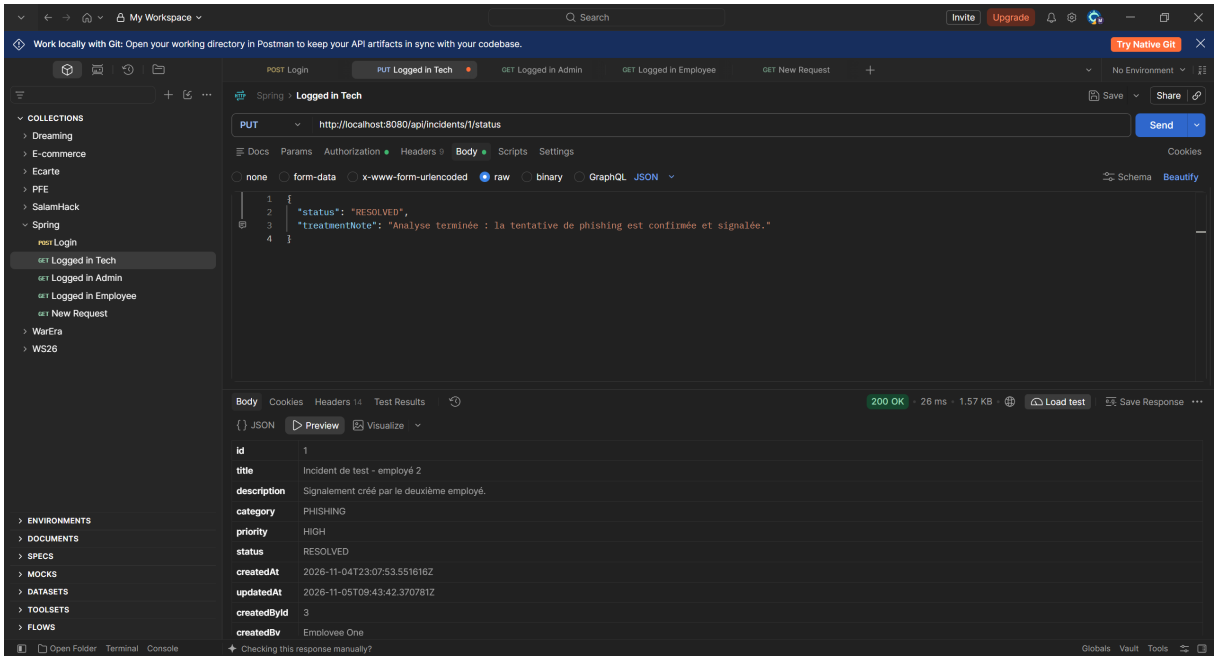


FIGURE 7.17 – Le technicien passe l'incident 1 à RESOLVED avec une note de traitement

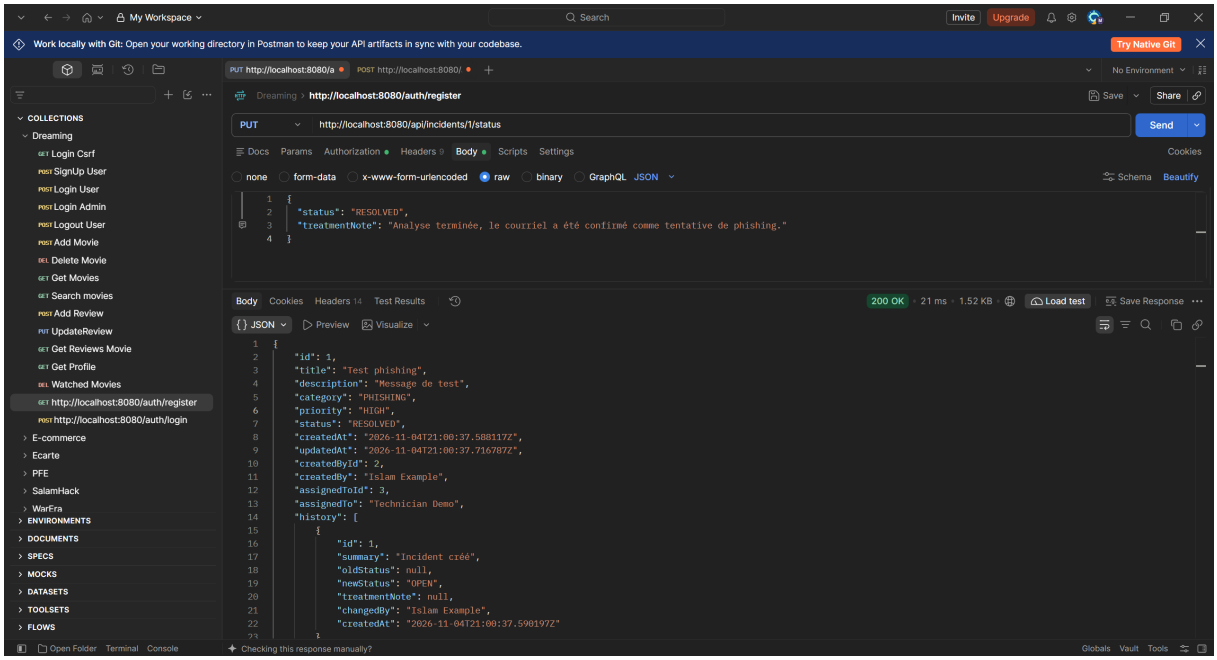


FIGURE 7.18 – Capture Postman de la résolution de l'incident par le technicien

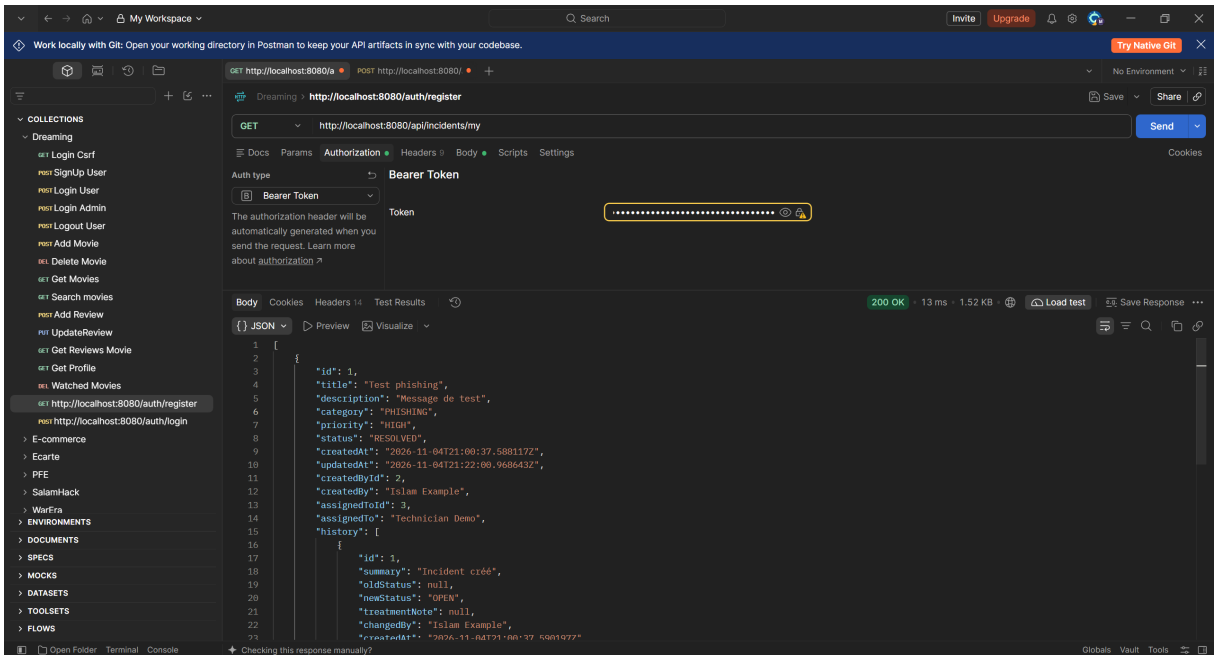


FIGURE 7.19 – Capture Postman de l'incident résolu visible par son créateur

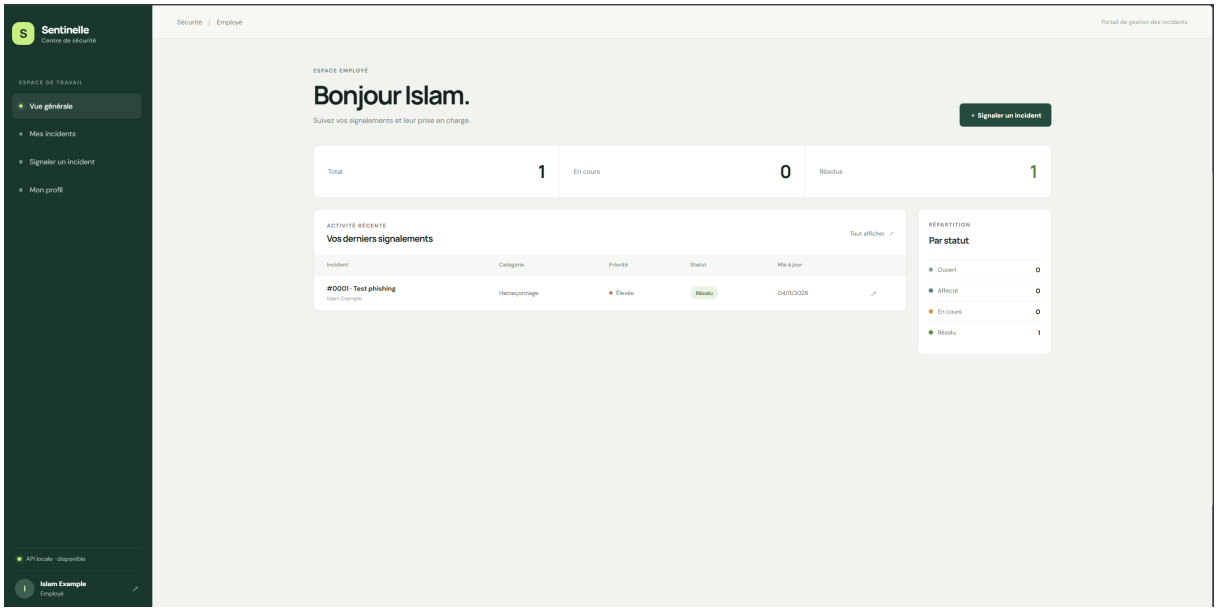


FIGURE 7.20 – Tableau de bord de l'employé avec le résumé de ses incidents

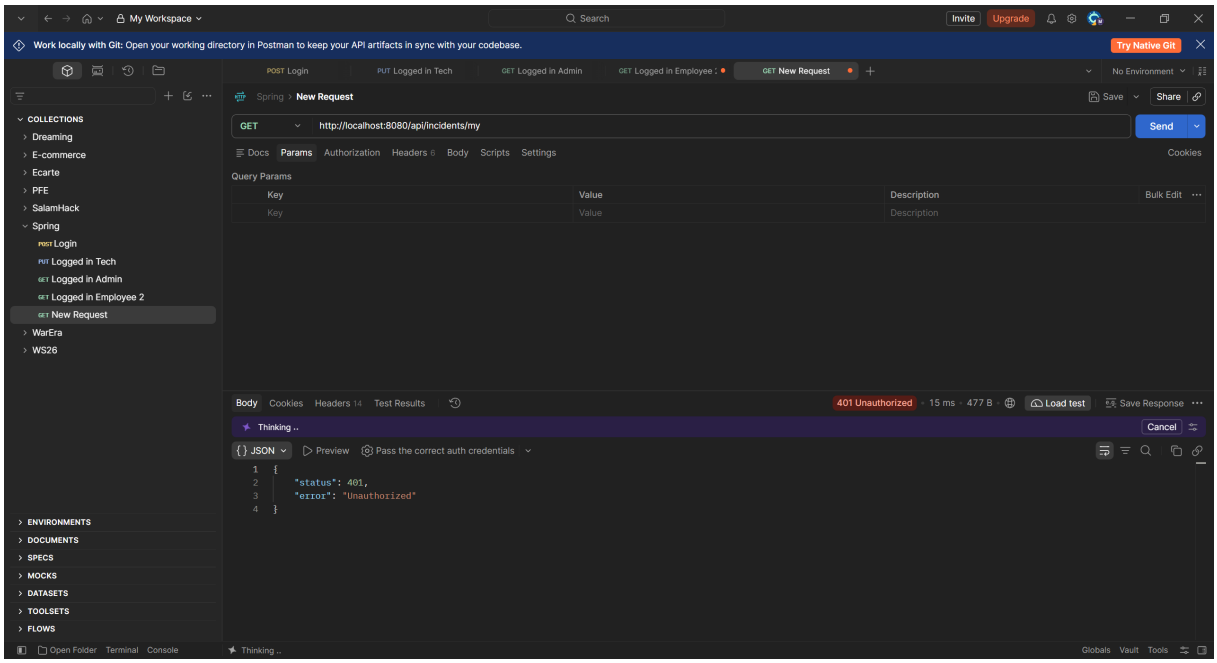


FIGURE 7.21 – Accès à une API protégée sans JWT

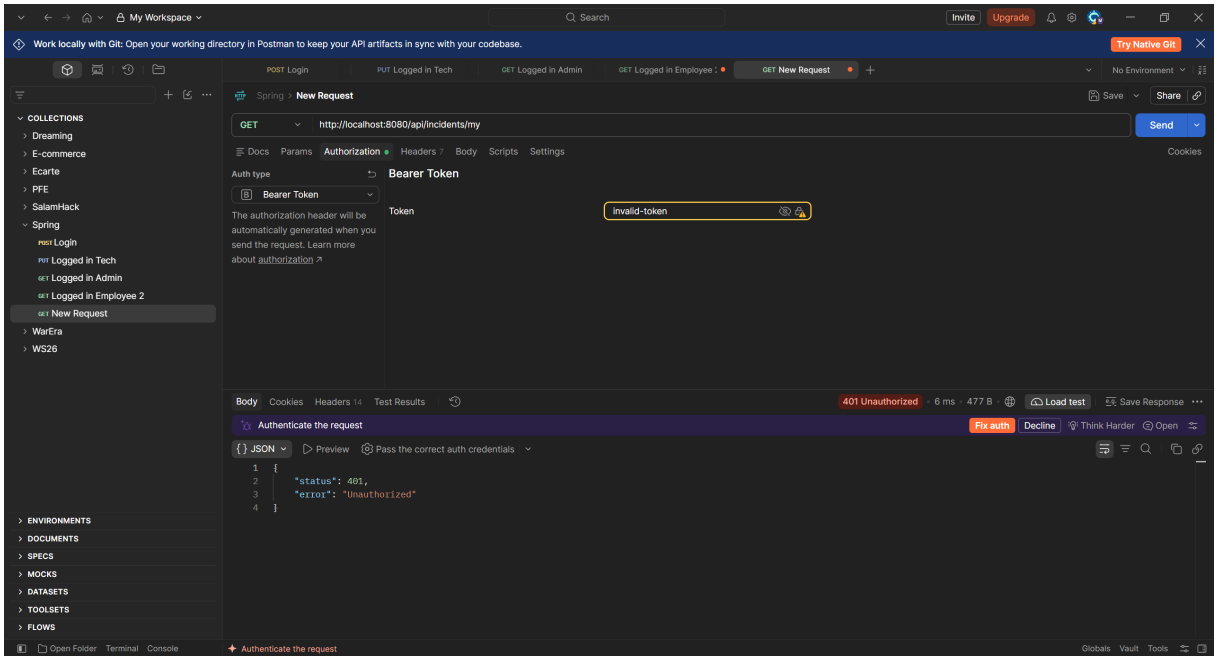


FIGURE 7.22 – Accès refusé avec un JWT invalide

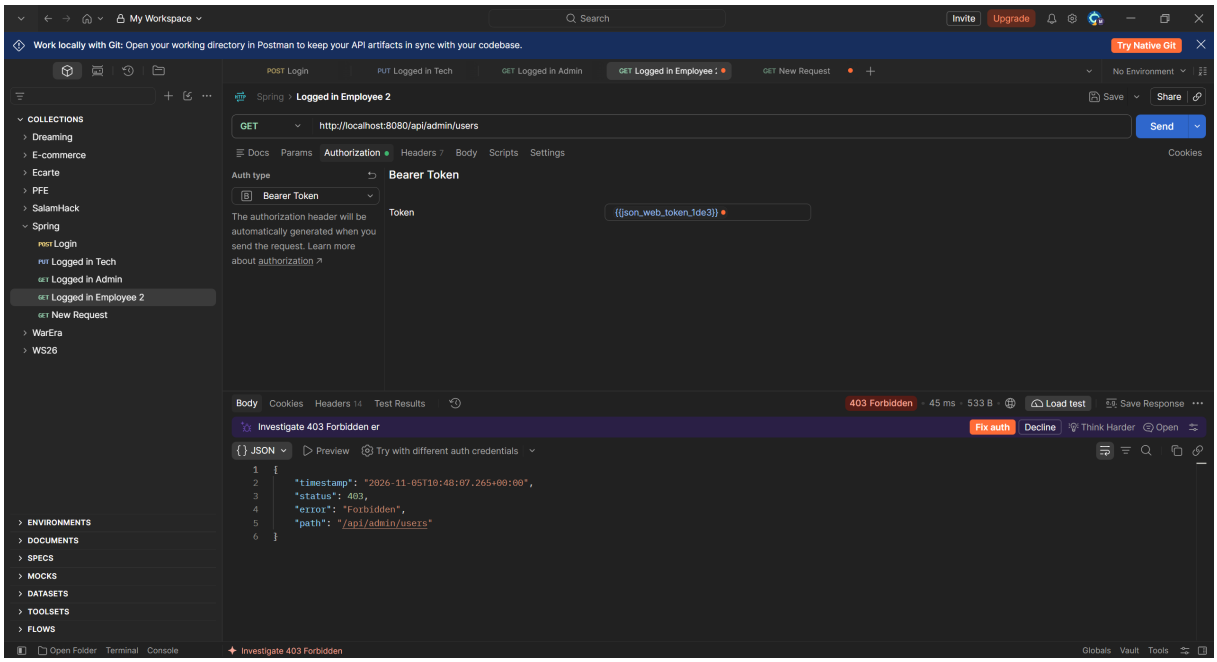


FIGURE 7.23 – Accès employé refusé à une route administrateur

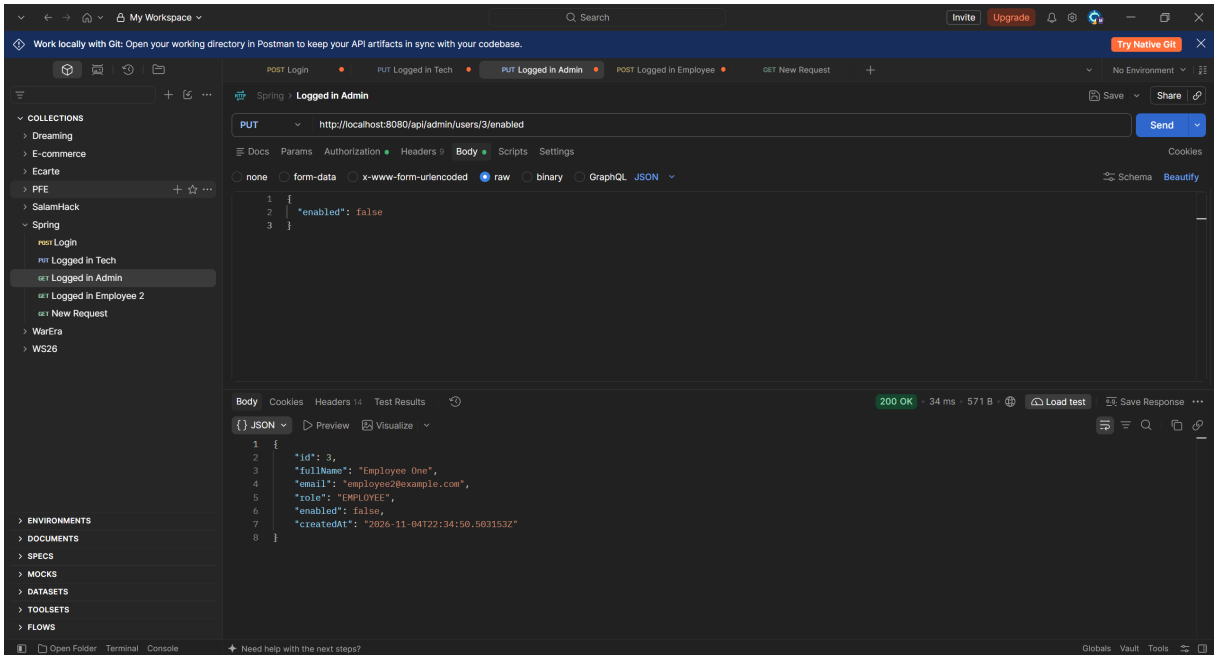


FIGURE 7.24 – Désactivation du compte par l'administrateur

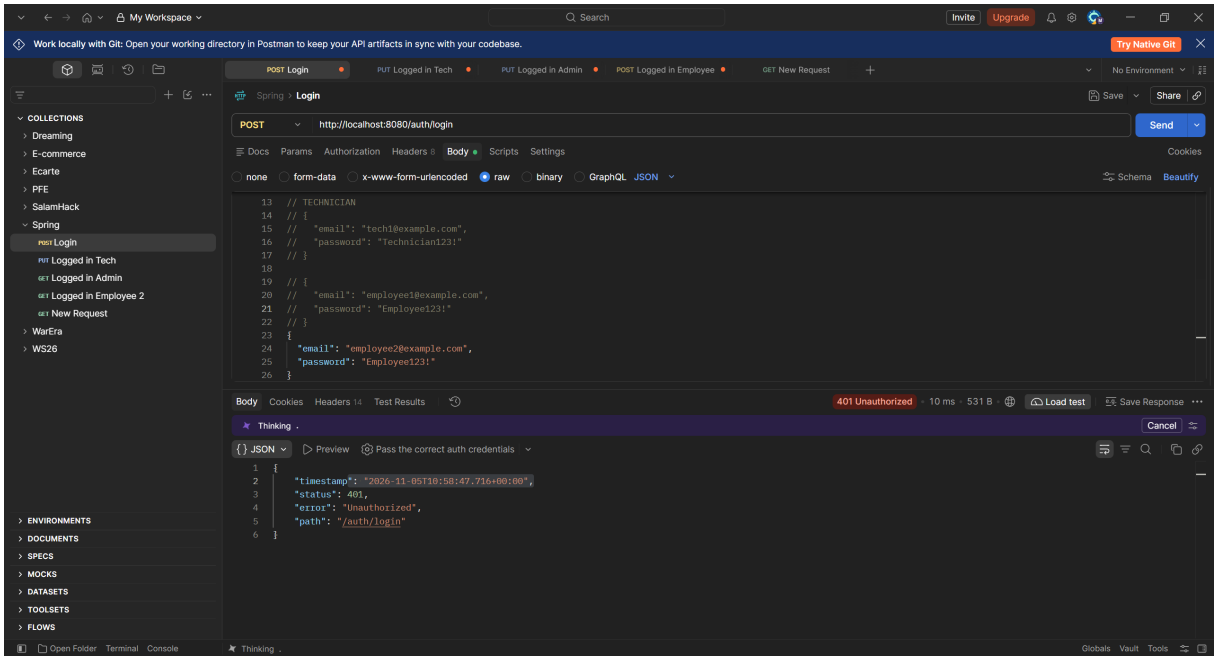


FIGURE 7.25 – Connexion refusée pour le compte désactivé

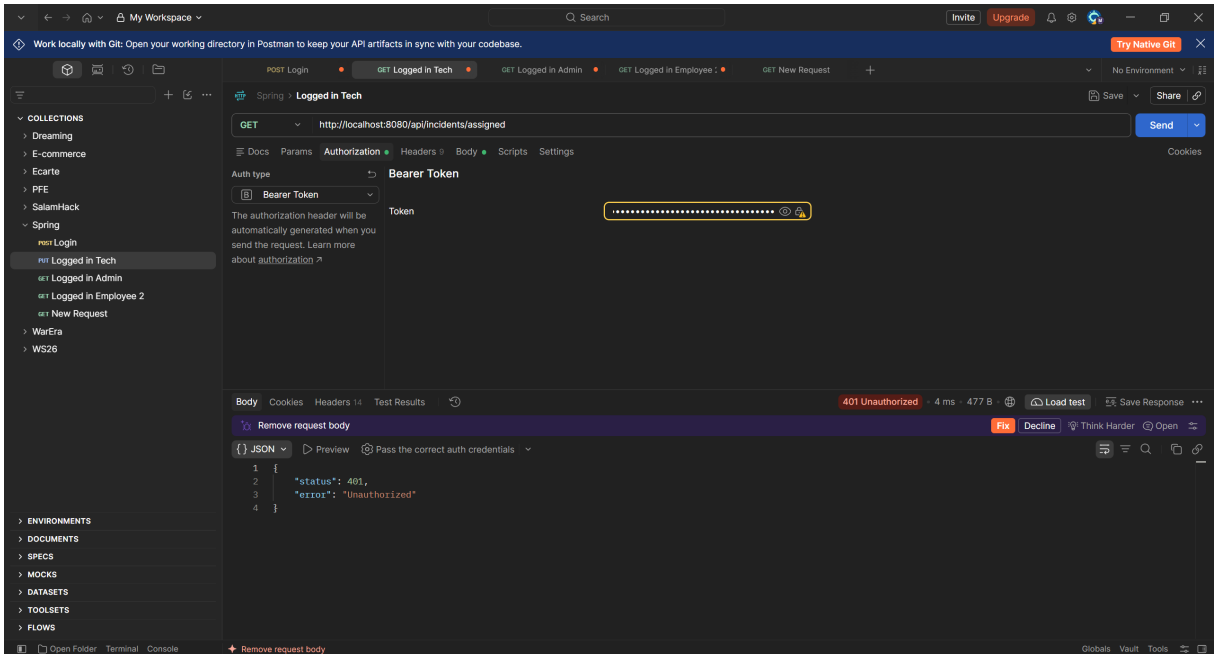


FIGURE 7.26 – Requête refusée avec un JWT expiré

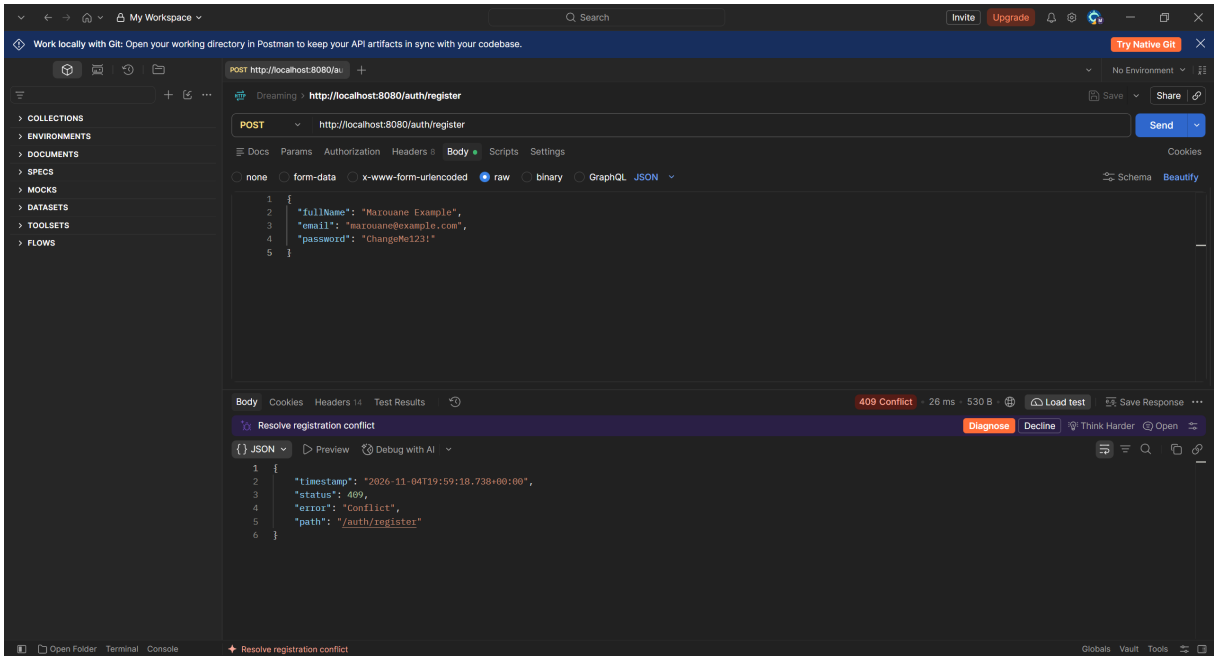


FIGURE 7.27 – Inscription refusée pour un email existant

8. Conclusion

L'application associe authentification JWT, contrôle des rôles et vérification de l'accès à chaque incident. Les historiques, notes et tableaux de bord assurent le suivi. La démonstration finale doit être effectuée avec les services MySQL, Spring Boot et React démarrés. Le renouvellement par refresh token, facultatif dans l'énoncé, n'est pas implémenté.